Lecture 6: Requirements Engineering 101

Vedaj Padman [*jp_vedaj @cb.amrita.edu*]

6.1 Introduction to Requirements Engineering

The development of any non-trivial software system begins long before design models are drawn or code is written. At the heart of this early phase lies the problem of understanding what the system is expected to achieve and the conditions under which it must operate. Historically, many software project failures can be traced not to poor implementation, but to incomplete, ambiguous, or misunderstood requirements. Requirements Engineering emerged as a systematic discipline to address this foundational challenge.

Requirements Engineering is motivated by the need to establish a clear and shared understanding between stakeholders and developers regarding the purpose, scope, and constraints of a software system. In complex organizational settings, stakeholders often possess differing, and sometimes conflicting, views of what the system should do. Without a structured approach to elicit, analyze, and document these expectations, development efforts risk converging on solutions that are technically sound but practically unusable.

Formally, Requirements Engineering is defined as the disciplined process of eliciting, analyzing, specifying, validating, and managing the requirements of a software system. It encompasses both technical and human-centered activities, recognizing that requirements originate from people, organizations, and operational contexts rather than from purely technical considerations.

The implications of effective Requirements Engineering are profound. Well-engineered requirements reduce rework, support realistic planning, and provide a stable foundation for design, implementation, and verification activities. Conversely, weak requirements propagate uncertainty throughout the lifecycle, amplifying costs and risks as the project progresses.

6.2 Nature and Types of Software Requirements

Understanding the nature of software requirements requires recognizing that not all requirements serve the same purpose. Software systems must provide specific services while simultaneously satisfying broader quality expectations and external constraints. Treating all requirements as homogeneous statements obscures their distinct roles and leads to incomplete specifications.

A software requirement may be defined as a documented condition or capability needed by a user to solve a problem, or required by a system to satisfy a contract, standard, or specification. This definition emphasizes both functional intent and compliance with external obligations.

Requirements are commonly categorized into functional and non-functional requirements. Functional requirements describe the services, behaviors, or functions that the system must provide. They focus on what the system does in response to inputs or events. Non-functional requirements, in contrast, specify quality attributes, performance expectations, usability considerations, reliability targets, and other constraints that shape how the system delivers its functionality.

In addition to this primary distinction, requirements may be further classified as domain requirements, which arise from the application domain itself, and constraints, which limit design or implementation choices due to regulatory, organizational, or technological factors. Each category contributes a different perspective on system expectations.

The implication of recognizing multiple requirement types is that a complete specification must balance behavioral correctness with quality, compliance, and feasibility. Overemphasis on functionality alone often leads to systems that technically work but fail in real-world usage.

6.3 The Requirements Engineering Process

The activities of Requirements Engineering are not isolated tasks but form a structured process that evolves alongside the system. This process exists to progressively reduce uncertainty while accommodating stakeholder feedback and contextual change.

The Requirements Engineering process typically comprises elicitation, analysis, specification, validation, and management. Elicitation focuses on discovering stakeholder needs through interviews, observation, and study of existing systems. Analysis refines these raw needs by resolving conflicts, prioritizing requirements, and assessing feasibility. Specification translates analyzed requirements into a structured and precise form suitable for communication and downstream development.

Validation ensures that documented requirements accurately reflect stakeholder intent and are internally consistent, complete, and testable. Management addresses the inevitable evolution of requirements by controlling changes and maintaining traceability across lifecycle artifacts.

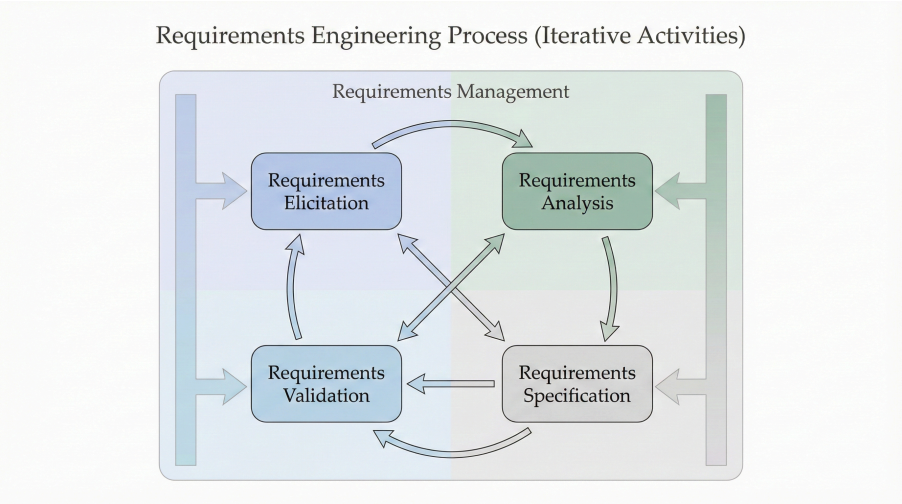


Figure 1: A conceptual view of the Requirements Engineering process and its iterative activities

The iterative nature of this process reflects the reality that understanding improves over time. Early requirements are often tentative and must be revisited as constraints, risks, and stakeholder priorities become clearer.

The implication of viewing Requirements Engineering as a process rather than a phase is that requirements-related decisions influence the system throughout its lifecycle. Continuous attention to requirements quality is essential to maintain alignment between the evolving system and its intended purpose.

6.4 Stakeholders and Their Role in Requirements

Requirements do not exist in isolation; they originate from individuals and groups with a stake in the system's success. Misidentifying or marginalizing stakeholders is a frequent source of incomplete or biased requirements.

A stakeholder is any individual, group, or organization that has an interest in, or is affected by, the system under development. Stakeholders may include end users, customers, managers, regulators,

developers, and maintenance personnel. Each stakeholder group brings unique concerns, priorities, and perspectives.

The role of stakeholders in Requirements Engineering is to articulate needs, validate interpretations, and negotiate trade-offs. Since stakeholders often possess implicit knowledge, effective communication techniques are essential to surface assumptions and clarify expectations.

The implication of stakeholder diversity is that Requirements Engineering is as much a socio-technical activity as a technical one. Success depends on balancing competing interests while preserving a coherent and achievable system vision.

6.5 Requirements Documentation and Specification

Capturing requirements in a durable and communicable form is a central outcome of Requirements Engineering. Informal understanding alone is insufficient for coordinating large development efforts or supporting verification activities.

A requirements specification is a structured document that describes the system's expected behavior, constraints, and qualities in a precise and unambiguous manner. It serves as a reference point for design, implementation, testing, and contractual agreement.

Effective specifications emphasize clarity, consistency, and verifiability. Natural language is commonly used, but it must be carefully controlled to avoid ambiguity. Supporting models and structured representations are often employed to improve precision and shared understanding.

The implication of rigorous documentation is that requirements become a stable artifact against which system evolution can be assessed. While specifications must remain adaptable, they provide essential continuity across teams and development stages.

Key Takeaways

- Requirements Engineering addresses the problem of understanding what a system should achieve.
- Poor requirements are a primary cause of software project failure.
- Requirements include functional, non-functional, domain, and constraint-related concerns.
- The Requirements Engineering process is iterative and continuous.
- Stakeholders play a central role in shaping and validating requirements.
- Requirements must be analyzed for feasibility and consistency.
- Documentation provides a shared and stable reference for development.
- Effective requirements reduce risk and rework across the lifecycle.

Key Terminology

Requirement A documented condition or capability needed by a user or system.

Functional Requirement A specification of system behavior or service.

Non-Functional Requirement A constraint or quality attribute affecting system operation.

Stakeholder An individual or group with an interest in the system.

Elicitation The activity of discovering stakeholder needs.

Specification The structured documentation of requirements.

Validation The process of ensuring requirements reflect true stakeholder intent.

Traceability The ability to link requirements across lifecycle artifacts.